K04 - The Four Directions: Search Without Borders

ctf: PINAKA CTF | **date**: 2026-11-01 | **category**: Forensics / Reversing | **flag_format**: PINAKA{...}

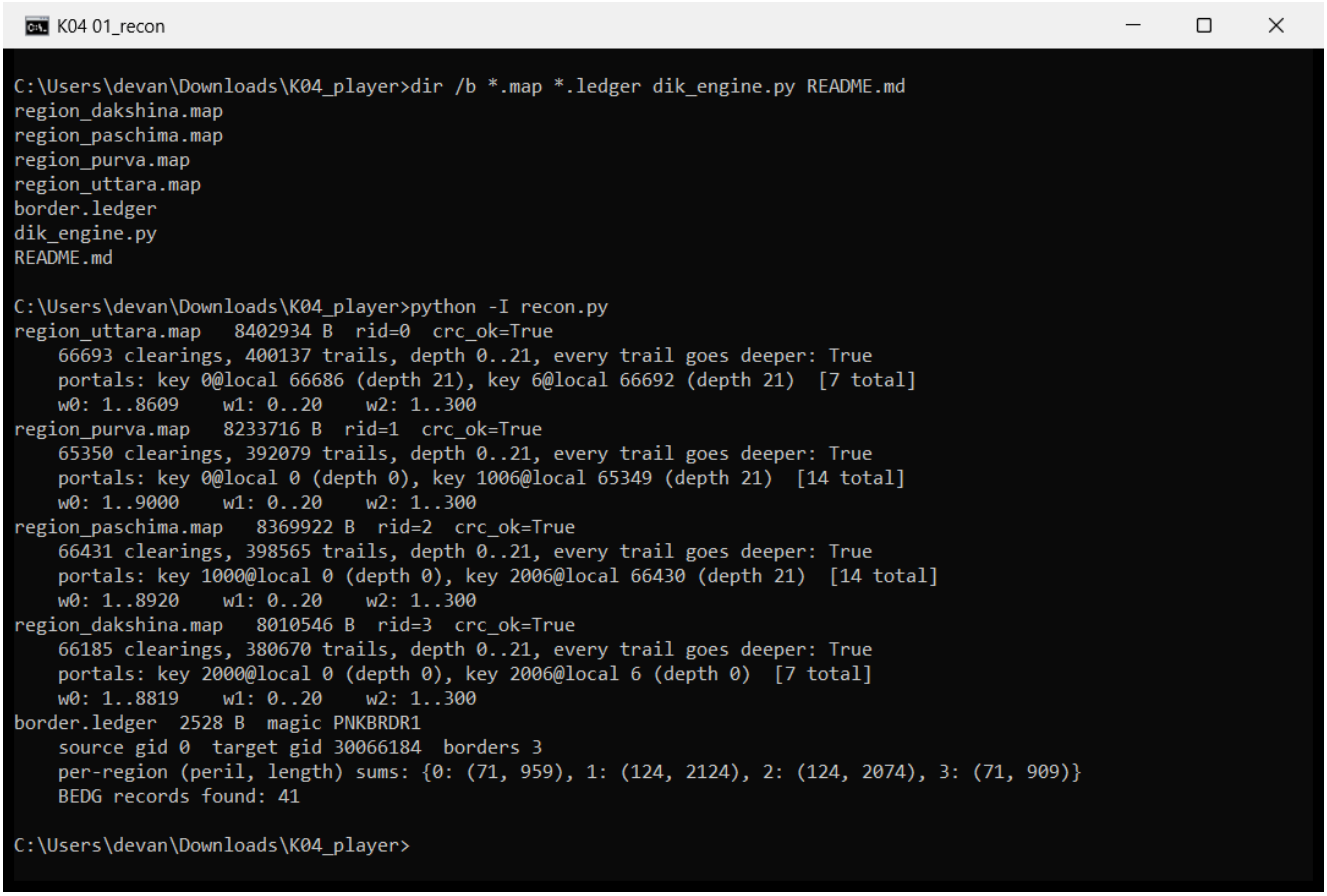
Flag: `PINAKA{kishkindha_dishamukha_sita_found}`

Summary

The challenge gives four region maps and one border ledger, all in undocumented binary formats, plus a small Python field engine that weighs routes inside a single region. To solve it I reversed both formats, read the routing metric out of the engine (`peril * 10^9 + length`), repaired the damaged ledger with a CRC brute force, and stitched the four regions into one graph of 1,571,472 trails. A Dijkstra search then gives exactly one best route from the muster point in the North to the target in the South, and the validator returns the flag for it.

Solved with help from Claude (Anthropic).

Step 1 - Recon



screenshots/01_recon.png - challenge files and the decoded map/ledger headers

`dik_engine.py` gives most of the map format away, because it parses the files itself:

Part	Layout
Header (36 B)	`PNKDISHA`, u16 version, u16 region id, u32 nodes, u32 trails, u32 portals, CRC32(nodes), CRC32(trails), CRC32(header)
Node (6 B)	u16 depth, i32 portal key (`-1` = ordinary clearing)
Trail (20 B)	u32 src, u32 dst, u32 w0, u32 w1, u32 w2

Every CRC checks out, so the maps are intact as the README says. Region ids are uttara=0 (N), purva=1 (E), paschima=2 (W), dakshina=3 (S). Some observations that mattered later:

- **Every trail goes to a deeper node**, so each region is a DAG with depths 0..21. That is why the engine can relax nodes in a single pass sorted by depth and still be exact.

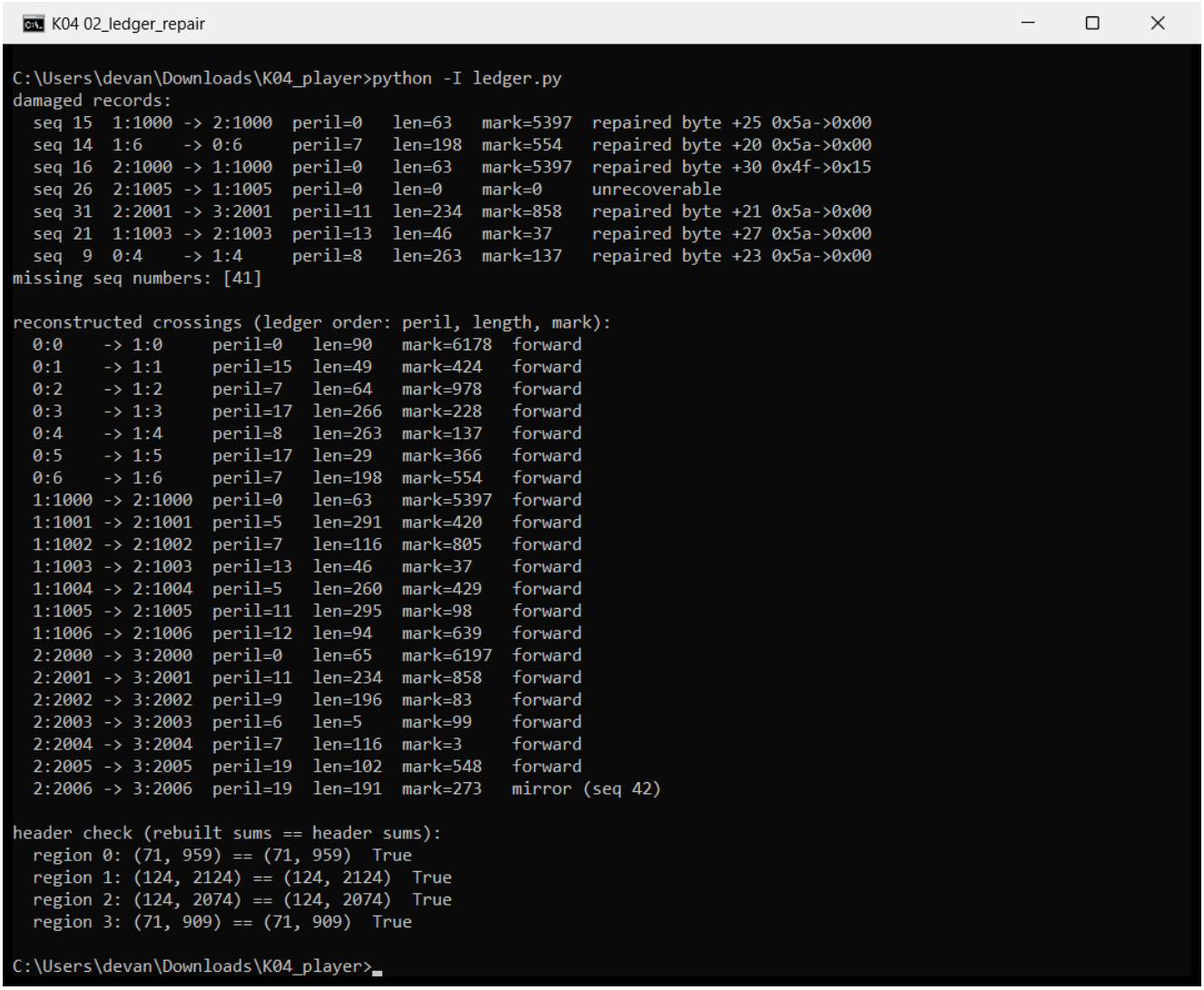
- **Portal keys line up across borders.** North exits carry keys 0-6 at depth 21; East has the same keys at depth 0 as entries and 1000-1006 as exits; West has 1000-1006 in and 2000-2006 out; South has 2000-2006 in.
- The value ranges hint at the field roles: w1 is 0..20, w2 is 1..300, and w0 goes up to about 9000.

The ledger (`PNKBRDR1`) has a short header, a per-region table and then `BEDG` records:

Part	Layout
Header	magic, u16 version, u16 regions, u16 borders, source (u16 rid, u32 local), target (u16 rid, u32 local), CRC32
Region table (22 B each)	u16 rid, u64 sum A, u64 sum B, CRC32
`BEDG` record (38 B)	magic, u16 body length (28), u32 seq, u16 src rid, u32 src key, u16 dst rid, u32 dst key, u32 a, u32 b, u32 c, CRC32 over magic+length+body

The header names the source (`0:0`, global id 0) and the target (`3:66184`, global id 30066184). Between the records there are runs of junk bytes from the "tearing"; they are high-entropy and carry nothing.

Step 2 - Repairing the border ledger



screenshots/02_ledger_repair.png - damaged records, CRC repairs, rebuilt crossings and the header cross-check

There are 41 `BEDG` records with sequence numbers 1..42, so seq 41 is lost entirely. Each crossing is written twice, once from each side of the border (`a -> b` and `b -> a`, with identical values), which is what makes reconstruction possible.

Seven records fail their CRC:

- Five have a single byte overwritten with `0x5A` (`'Z'`), e.g. a destination key of `0x5A0007D1` instead of 2001. Trying every value for every byte offset gives exactly one CRC-valid fix each time, always `0x5A -> 0x00`.
- Seq 16 has a changed value byte (`0x4F`); the unique CRC fix turns it back into the same value its mirror (seq 15) holds.
- Seq 26 is zeroed and cannot be fixed, but its mirror (seq 25) is intact.
- The lost seq 41 (`2:2006 -> 3:2006`) is rebuilt from its mirror, seq 42.

The region table in the header confirms the field roles. For each region, the two sums equal the sum of field `a` and the sum of field `b` over all crossings touching that region (e.g. North: 71 and 959). So in the ledger, **`a` is peril and `b` is length**, and `c` is the same kind of decoy value as `w0` in the maps. The ledger stores its fields in a different order from the map trails, which is the first trap. All four regions match after the repair.

The second trap is direction. The mirrored copies are bookkeeping, not extra roads: a route from North to South only ever travels forward, so the stitched graph gets **one directed crossing per portal pair (21 in total)**, N->E, E->W, W->S. The validator's per-field feedback (it lists which fields are wrong) confirmed both the count of 21 and the field order.

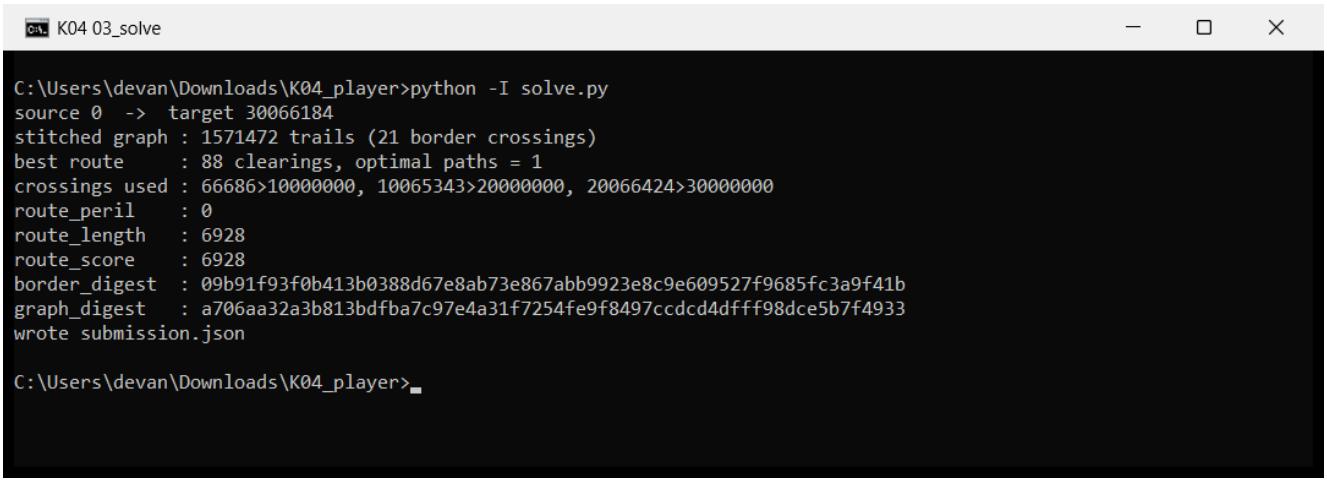
Step 3 - Metric, stitching and the route

The engine's toll function is the whole metric:

```
def _toll(self, f0, f1, f2):
    a, b, c = f0, f1, f2
    return (b * _K) + c          # _K = 0x3B9ACA00 = 1_000_000_000
```

So `w0` (the "first number" the README warns about) is ignored, **w1 is peril, w2 is length, and peril is weighted by 10^9** so that it always dominates length. Comparing routes by `peril * 10^9 + length` is the same as comparing (peril, length) lexicographically.

Global ids are `region_id * 10_000_000 + local_id`. Portal keys from the ledger map to local ids through the node table. With all intra-region trails plus the 21 crossings, a Dijkstra search (which also counts tied paths) from `0` to `30066184` gives one optimal route:



screenshots/03_solve.png - solve.py building the stitched graph and printing the answer

- 88 clearings, exactly one optimal path
- crossings at keys 0 (N->E), 1000 (E->W) and 2000 (W->S)
- `route_peril = 0`, `route_length = 6928`, `route_score = 6928`
- `committed_edge_count = 1571472`, `border_edge_count = 21`

The two digests are SHA-256 over the sorted, `;`-joined strings `src>dst:peril:length`: over the 21 crossings for `border_digest`, and over every trail for `committed_graph_digest`.

One detail: the zero-peril route runs through one extra trail appended at the very end of each map (a parallel copy of an existing trail, with peril 0 and a large `w0`) and through the three zero-peril crossings, which also carry large decoy values. Anyone who ranks by the first field steers away from exactly this path.

The full solve script:

```
#!/usr/bin/env python3
"""K04 - The Four Directions: parse the four dishā-maps, repair the border
ledger, stitch the regions into one graph and trace the best route.

    python -I solve.py     -> prints the answer and writes submission.json
"""
import hashlib
import heapq
import json
import os
import struct
import zlib

HERE = os.path.dirname(os.path.abspath(__file__))
REGIONS = ["uttara", "purva", "paschima", "dakshina"]   # rid 0..3 = N, E, W, S
K = 1_000_000_000                                       # peril weight (dik_engine._K)
# magic ver rid nodes trails portals crc_nodes crc_trails crc_hdr
MAP_HDR = "<8sHHIIIIII"
# seq src_rid src_key dst_rid dst_key peril length mark
REC = "<IHIHIIII"


def crc(b):
    return zlib.crc32(b) & 0xffffffff


def gid(rid, local):
    return rid * 10_000_000 + local


def path(name):
    return os.path.join(HERE, name)


# ---------------------------------------------------------------- dishā-map
def parse_map(fn):
    d = open(fn, "rb").read()
    h = struct.calcsize(MAP_HDR)
    magic, ver, rid, nc, ec, pc, ncrc, ecrc, hcrc = struct.unpack(MAP_HDR, d[:h])
    nb = d[h:h + nc * 6]
    eb = d[h + nc * 6:h + nc * 6 + ec * 20]
    ok = (magic == b"PNKDISHA" and hcrc == crc(d[:h - 4])
          and crc(nb) == ncrc and crc(eb) == ecrc)
    # node = (depth, portal_key), trail = (src, dst, w0, w1, w2)
    nodes = [struct.unpack_from("<Hi", nb, i * 6) for i in range(nc)]
    trails = [struct.unpack_from("<IIIII", eb, j * 20) for j in range(ec)]
    portals = {pk: i for i, (_, pk) in enumerate(nodes) if pk != -1}
    return dict(rid=rid, nodes=nodes, trails=trails, portals=portals,
                portal_count=pc, size=len(d), crc_ok=ok)


def load_maps():
    return [parse_map(path(f"region_{n}.map")) for n in REGIONS]


# ---------------------------------------------------------------- border ledger
def parse_ledger(fn):
    d = open(fn, "rb").read()
    assert d[:8] == b"PNKBRDR1"
    ver, nreg, nborders, src_r, src_l, dst_r, dst_l = struct.unpack_from("<HHHHIHI", d, 8)
    assert struct.unpack_from("<I", d, 0x1a)[0] == crc(d[:0x1a])
    sums, off = {}, 0x1e
    for _ in range(nreg):                       # rid, sum(peril), sum(length), crc
        rid, sp, sl, c = struct.unpack_from("<HQQI", d, off)
        assert c == crc(d[off:off + 18])
        sums[rid] = (sp, sl)
        off += 22
    records, i = [], off
    while (j := d.find(b"BEDG", i)) >= 0:
        raw = d[j:j + 38]
        stored = struct.unpack_from("<I", raw, 34)[0]
```

```
        records.append(dict(off=j, raw=raw, stored_crc=stored))
        i = j + 4
    return dict(source=gid(src_r, src_l), target=gid(dst_r, dst_l),
                nborders=nborders, sums=sums, records=records)


def repair(rec):
    """Return (fields, status). Bad records get a one-byte CRC brute force."""
    raw = bytearray(rec["raw"])
    if crc(raw[:34]) == rec["stored_crc"]:
        return struct.unpack_from(REC, raw, 6), "ok"
    fixes = []
    for p in range(4, 34):
        orig = raw[p]
        for b in range(256):
            raw[p] = b
            if b != orig and crc(raw[:34]) == rec["stored_crc"]:
                fixes.append((p, orig, b, struct.unpack_from(REC, raw, 6)))
        raw[p] = orig
    if len(fixes) == 1:
        p, o, b, f = fixes[0]
        return f, f"repaired byte +{p} 0x{o:02x}->0x{b:02x}"
    return struct.unpack_from(REC, raw, 6), "unrecoverable"


def crossings(led):
    """One directed crossing per portal pair, in travel order N->E->W->S.
    Each crossing is written twice in the ledger (once from each side), so a
    lost or unrecoverable copy is rebuilt from its mirror."""
    good = {}
    for rec in led["records"]:
        f, status = repair(rec)
        if status != "unrecoverable":
            seq, sr, sk, dr, dk, peril, length, mark = f
            good[(sr, sk, dr, dk)] = (peril, length, mark, seq, status)
    out = []
    for r, keys in ((0, range(0, 7)), (1, range(1000, 1007)), (2, range(2000, 2007))):
        for k in keys:
            fwd, rev = good.get((r, k, r + 1, k)), good.get((r + 1, k, r, k))
            v = fwd or rev
            via = "forward" if fwd else f"mirror (seq {rev[3]})"
            out.append(dict(src=(r, k), dst=(r + 1, k), peril=v[0], length=v[1],
                            mark=v[2], via=via))
    return out


def check_sums(led, xs):
    got = {r: [0, 0] for r in led["sums"]}
    for x in xs:
        for r in (x["src"][0], x["dst"][0]):
            got[r][0] += x["peril"]
            got[r][1] += x["length"]
    return {r: (tuple(got[r]), led["sums"][r]) for r in led["sums"]}


# ---------------------------------------------------------------- stitching
def stitch(maps, xs):
    edges = []                                  # (src_gid, dst_gid, peril, length)
    for m in maps:
        r = m["rid"]
        edges += [(gid(r, s), gid(r, t), w1, w2) for s, t, w0, w1, w2 in m["trails"]]
    border = [(gid(x["src"][0], maps[x["src"][0]]["portals"][x["src"][1]]),
               gid(x["dst"][0], maps[x["dst"][0]]["portals"][x["dst"][1]]),
               x["peril"], x["length"]) for x in xs]
    return edges + border, border


def digest(edges):
    return hashlib.sha256(";".join(sorted(f"{s}>{t}:{p}:{l}" for s, t, p, l in edges))
                          .encode()).hexdigest()


def best_route(edges, src, dst):
```

```
    adj = {}
    for s, t, p, l in edges:
        adj.setdefault(s, []).append((t, p, l))
    best, prev, ways, done = {src: 0}, {src: None}, {src: 1}, set()
    pq = [(0, src)]
    while pq:
        c, u = heapq.heappop(pq)
        if u in done:
            continue
        done.add(u)
        for v, p, l in adj.get(u, ()):
            nc = c + p * K + l
            if v not in best or nc < best[v]:
                best[v], prev[v], ways[v] = nc, (u, p, l), ways[u]
                heapq.heappush(pq, (nc, v))
            elif nc == best[v]:
                ways[v] += ways[u]
    route, peril, length, v = [dst], 0, 0, dst
    while prev[v]:
        u, p, l = prev[v]
        route.append(u)
        peril += p
        length += l
        v = u
    return route[::-1], peril, length, ways[dst]

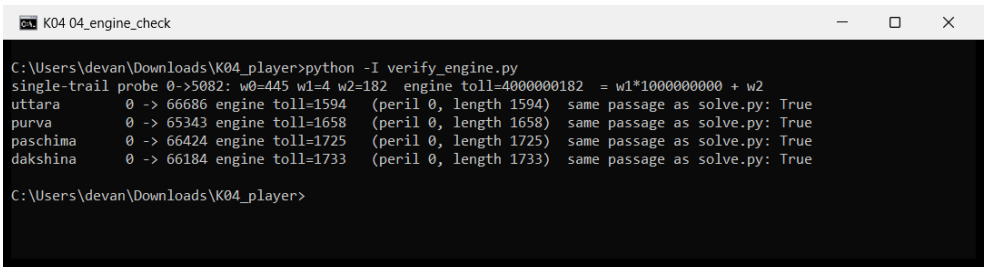

def main():
    maps = load_maps()
    led = parse_ledger(path("border.ledger"))
    xs = crossings(led)
    for r, (got, want) in check_sums(led, xs).items():
        assert got == want, f"region {r}: ledger sums {got} != header {want}"
    edges, border = stitch(maps, xs)
    route, peril, length, ways = best_route(edges, led["source"], led["target"])

    sub = dict(route=route, route_peril=peril, route_length=length,
               route_score=peril * K + length,
               metric=dict(peril_field="w1", length_field="w2", peril_weight=K),
               committed_edge_count=len(edges), border_edge_count=len(border),
               border_digest=digest(border), committed_graph_digest=digest(edges))
    json.dump(sub, open(path("submission.json"), "w"))

    hops = [(a, b) for a, b in zip(route, route[1:])
            if a // 10_000_000 != b // 10_000_000]
    print(f"source {led['source']}  ->  target {led['target']}")
    print(f"stitched graph : {len(edges)} trails ({len(border)} border crossings)")
    print(f"best route     : {len(route)} clearings, optimal paths = {ways}")
    print(f"crossings used : " + ", ".join(f"{a}>{b}" for a, b in hops))
    print(f"route_peril    : {peril}")
    print(f"route_length   : {length}")
    print(f"route_score    : {peril * K + length}")
    print(f"border_digest  : {sub['border_digest']}")
    print(f"graph_digest   : {sub['committed_graph_digest']}")
    print("wrote submission.json")


if __name__ == "__main__":
    main()
```

Step 4 - Verifying against the field engine

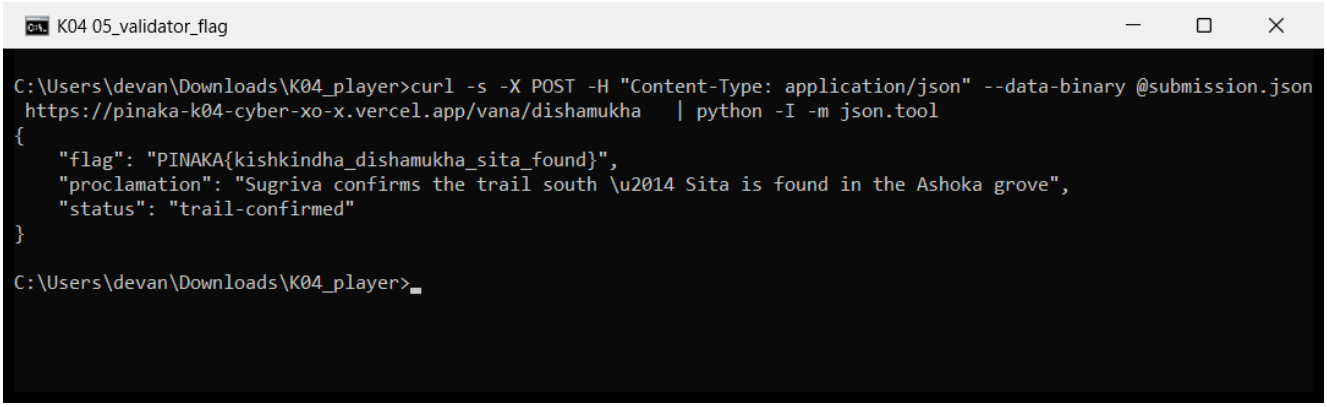


screenshots/04_engine_check.png - the provided dik_engine agrees with the solver in every region

As an independent check I ran the challenge's own `dik_engine.Yantra`:

- A single trail out of the muster point (`0 -> 5082`, w0=445, w1=4, w2=182) has toll `4000000182`, which is exactly `w1 * 10^9 + w2`, with w0 playing no part.
- For each region, the engine's choice between the route's entry and exit portals is the **same passage** my solver picked, with peril 0 and lengths 1594 + 1658 + 1725 + 1733. Adding the three crossings (90 + 63 + 65) gives 6928, matching `route_length`.

Step 5 - Getting the flag



screenshots/05_validator_flag.png - POSTing submission.json to the validator

`solve.py` writes the full answer to `submission.json`, which goes to `POST /vana/dishamukha`. The validator checks every field together and confirms the trail.

Flag

```
PINAKA{kishkindha_dishamukha_sita_found}
```

Files

File	Purpose
`solve.py`	Parses maps and ledger, repairs the ledger, stitches the graph, finds the route, writes `submission.json`
`recon.py`	Prints the decoded headers, CRC status, depth/DAG check, portals and field ranges
`ledger.py`	Lists damaged ledger records, the CRC repairs, the 21 rebuilt crossings and the header-sum check
`verify_engine.py`	Cross-checks the route against the provided `dik_engine.py`
`submission.json`	The answer sent to the validator
`build_pdf.py`	Builds `K04_writeup.pdf` from this file and the screenshots
`screenshots/01_recon.png`	Recon
`screenshots/02_ledger_repair.png`	Ledger repair
`screenshots/03_solve.png`	Solve
`screenshots/04_engine_check.png`	Engine verification
`screenshots/05_validator_flag.png`	Validator response with the flag